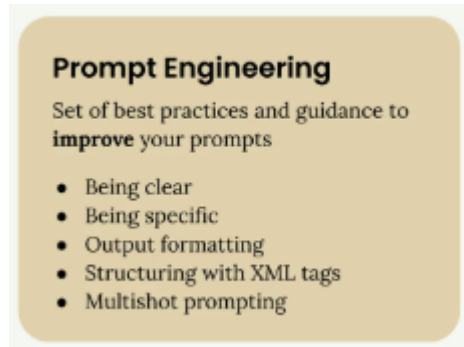


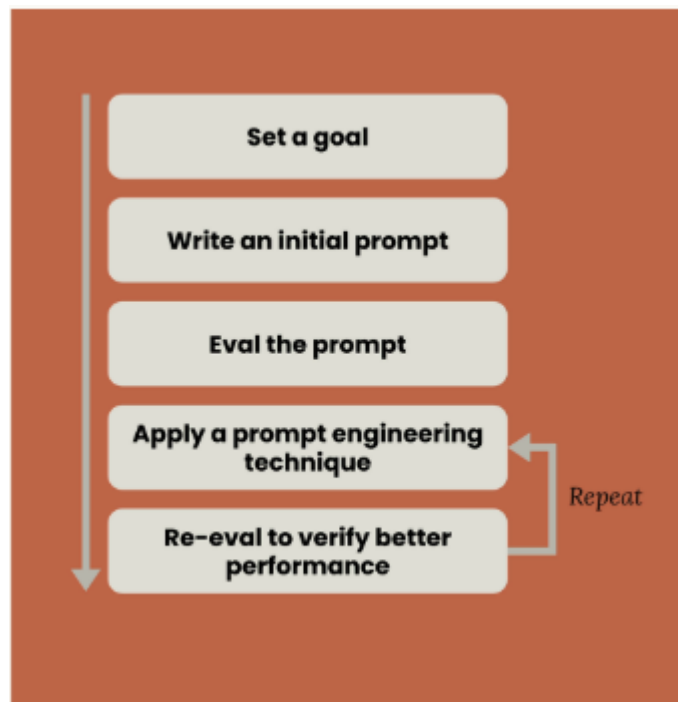
Prompt Engineering

Prompt engineering is about taking a prompt you've written and improving it to get more reliable, higher-quality outputs.

This process involves iterative refinement - starting with a basic prompt, evaluating its performance, then systematically applying engineering techniques to improve it.



Steps



Being Clear and Direct

When crafting that crucial first line, you want to focus on two key principles: clarity and directness. This means using simple language that leaves no room for ambiguity about what you want Claude to do.

Be clear and direct

"Clear"

- Use simple language
- State what you want explicitly
- Lead your prompt with a simple statement of the model's task

Instead of: "I need to know about those things people put on their roofs that use sun - those solar panel things, I think they're called"

Use: "Write three paragraphs about how solar panels work"

"Direct"

- Use instructions, not questions
- Use direct action verbs ("Write", "Create", "Generate")

Instead of: "I was reading about renewable energy and geothermal energy sounds neat. What countries use it?"

Use: "Identify three countries that use geothermal energy. Include generation stats for each."

Being Specific

When working with Claude, one of the most effective ways to improve your results is to be specific about what you want. Instead of leaving everything up to the model's interpretation, you can provide clear guidelines or steps that direct Claude toward the kind of output you're looking for.

Think about it this way: if you ask Claude to "write a short story about a character who discovers a hidden talent," Claude could go in countless directions. The story might be 200 words or 2,000 words. It might have one character or five. It could focus on any type of talent discovery scenario.

Be specific

Provide a list of guidelines or steps to direct the model

Not Great

```
prompt = f"""
Write a short story about a
character who discovers a hidden
talent
"""
```

Better!

```
prompt = f"""
Write a short story about a character who
discovers a hidden talent

Guidelines:
1. Keep the story under 1,000 words
2. Include a clear action that reveals the
character's talent
3. Include at least one supporting
character
"""
```

By adding specific guidelines, you give Claude a clearer target to aim for. This dramatically improves both the consistency and quality of the output.

Two Types of Guidelines

There are two main approaches to being specific in your prompts, and you'll often see them used together in professional applications.

Guideline Types

List qualities that the output should have

Write a short story about a character who discovers a hidden talent

Guidelines:

1. Keep the story under 1,000 words
2. Include a clear action that reveals the character's talent
3. Include at least one supporting character

Provide steps the model should follow

Write a short story about a character who discovers a hidden talent

Follow these steps:

1. Brainstorm 3 talents that would create dramatic tension
2. Pick the most interesting talent
3. Outline a pivotal scene that reveals the talent
4. Brainstorm 3 supporting characters types that could increase the impact of this discovery

Output Quality Guidelines

The first type focuses on listing qualities that your output should have. These guidelines help you control:

- Length of the response
- Structure and format
- Specific attributes or elements to include
- Tone or style requirements

For example, you might specify that a story should be under 1,000 words, include a clear action that reveals the character's talent, and feature at least one supporting character.

When to Use Each Approach

Here's a practical guide for when to use each type of specificity

1. Always Use Output Guidelines

You should include quality guidelines in almost every prompt you write. They're your safety net for getting consistent, useful results.

2. Use Process Steps for Complex Problems

Add step-by-step instructions when dealing with:

- Troubleshooting complex problems
- Decision-making scenarios
- Critical thinking tasks
- Any situation where you want Claude to consider multiple angles

When to Provide “Steps”

Try listing steps for the model to follow with these kinds of prompts:

- Troubleshooting hard problems
- Decision making
- Critical thinking
- Anytime you want to force Claude to consider a “wider” view

```
Write a one page decision report to troubleshoot why a sales team's numbers have dropped 30% last quarter.
```

```
Follow these steps:
```

1. Compare current vs previous market metrics
2. Identify relevant industry changes
3. Analyze individual team member performance
4. Consider recent organizational changes
5. Review customer feedback

For instance, if you're asking Claude to analyze why a sales team's performance dropped, you'd want to guide it through examining market metrics, industry changes, individual performance, organizational changes, and customer feedback - rather than letting it focus on just one potential cause.

Combining Both Approaches

In professional prompting, you'll often see both techniques used together. You might have guidelines that control the format and content of your output, plus steps that ensure Claude thinks through the problem thoroughly before responding.

This combination gives you both consistency in your results and confidence that Claude has considered all the important factors in reaching its conclusion.

Structure with XML Tags

When you're building prompts that include a lot of content, Claude can sometimes struggle to understand which pieces of text belong together or what different sections are supposed to represent.

XML tags provide a simple way to add structure and clarity to your prompts, especially when you're interpolating large amounts of data.

Why Structure Matters

Consider a prompt where you need to analyze 20 pages of sales records. Without clear boundaries, Claude might have trouble distinguishing between your instructions and the actual data you want analyzed.

Providing Structure

Use XML tags to separate distinct portions of the prompt

- Most useful when including a lot of context
- Help serve as delimiters for Claude

ANTHROPIC

Write a one page decision report to troubleshoot why a sales team's numbers have dropped 30% last quarter.

Here are the last 20 pages of our sales records:
{sales_records}

Follow these steps:

1. Compare current vs previous market metrics
2. Identify relevant industry changes
3. Analyze individual team member performance
4. Consider recent organizational changes
5. Review customer feedback

The example above shows how unclear boundaries can make it difficult for Claude to parse your intent. By wrapping the sales records in XML tags like `<sales_records>` and `</sales_records>`, you create clear delimiters that help Claude understand the structure of your prompt.

```
<sales_records>
...
</sales_records>
```

you create clear delimiters that help Claude understand the structure of your prompt.

Providing Structure

Use XML tags to separate distinct portions of the prompt

- Most useful when including a lot of context
- Help serve as delimiters for Claude

ANTHROPIC

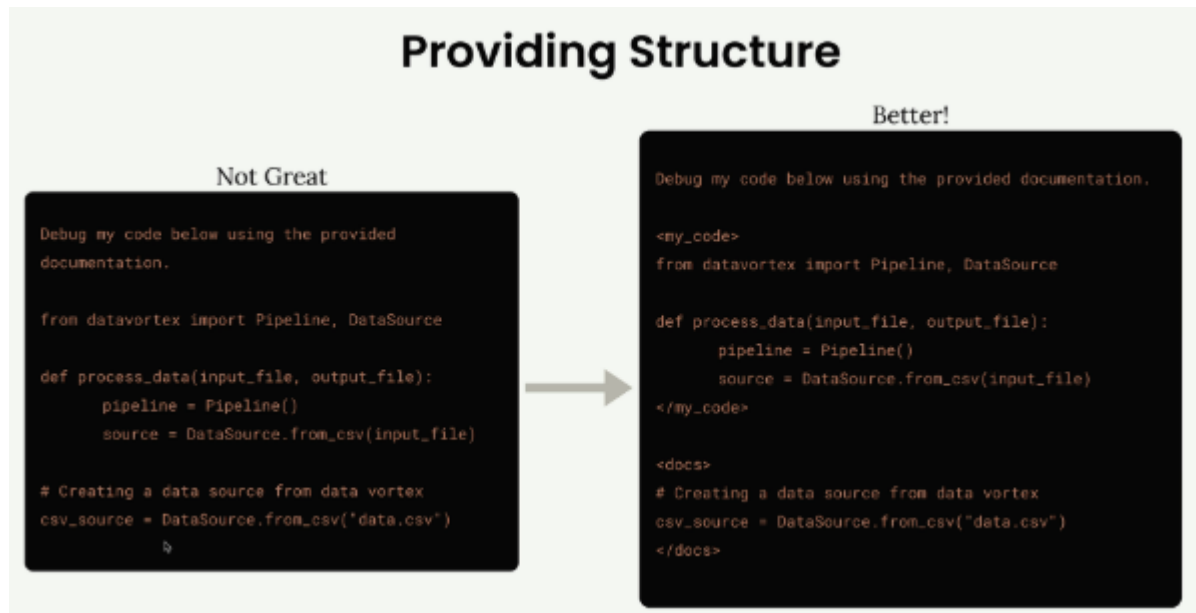
Write a one page decision report to troubleshoot why a sales team's numbers have dropped 30% last quarter.

Here are the last 20 pages of our sales records:
<sales_records>
{sales_records}
</sales_records>

Follow these steps:

1. Compare current vs previous market metrics
2. Identify relevant industry changes
3. Analyze individual team member performance
4. Consider recent organizational changes
5. Review customer feedback

Here's a more dramatic example of why XML tags matter. If you ask Claude to debug code using provided documentation, mixing everything together creates confusion:



The "Not Great" version makes it nearly impossible to tell what's code versus documentation. The "Better" version uses `<my_code>` and `<docs>` tags to create clear boundaries.
